

Entrega 2:

AppAdmin, AppUsuarios y Login+Apikey

Dispositivos e Infraestructuras de Sistema Multimedia- G2
Grado de Ingeniería Multimedia

REALIZADO POR:

Abel Martinez Molina

3 de diciembre de 2025

Puesta en Marcha de la App:..... 3

Requisitos:.....3

Backend:..... 3

Login:.....3

AppAdmin:.....3

AppUsuarios:.....4

Lógica de AppAdmin:..... 4

Lógica de AppUsuarios:.....4

Lógica del Backend:..... 5

Apikey:..... 5

Login:..... 5

Puesta en Marcha de la App:

Requisitos:

- Node.js 18+
- npm
- MySQL con la base de datos del proyecto (tabla `Usuarios`, `Fichajes`, `Trabajos`, etc.) y columna `permisos` en `Usuarios`.
- macOS con zsh (comandos aquí están pensados para zsh).

Backend:

Para levantar el servidor Express basado en OpenApi y poder hacer correctamente las llamadas al Api se debe ejecutar los siguientes comandos desde la terminal:

```
``zsh
pushd /Users/abel/DISM/Practical/api-rest-openApi/generated
npm install
npm start
``
```

Login:

El login está diseñado para que dependiendo el valor del atributo “permisos” de la tabla usuarios dirija a AppUsuarios o AppAdmin. Se ubica en <http://localhost:8080> y para levantarlo:

```
``zsh
pushd /Users/abel/DISM/Practical/login
npm install
PORT=8080 npm start
``
```

AppAdmin:

Contiene el código para el developer o el usuario con privilegios de administrador. Se sirve en <http://localhost:8100> con:

```
``zsh
pushd /Users/abel/DISM/Practical/AppAdmin
ionic serve
``
```

AppUsuarios:

Contiene el código para los usuarios y trabajadores. Se sirve en <http://localhost:8101> con:

```
```zsh
pushd /Users/abel/DISM/Practical/AppUsuarios
ionic serve
```
```

Lógica de AppAdmin:

Se compone principalmente de 3 páginas, otra extra para el mapa y el inicio.

La página **`gestion-usuarios`** se encarga de hacer GET, POST, PUT y DELETE sobre la tabla Usuarios a través de los métodos de ApiService

La página **`gestion-trabajos`** se encarga de hacer GET, POST, PUT y DELETE sobre la tabla Usuarios a través de los métodos de ApiService

La página de **`gestion-fichajes`** permite cargar fichajes. Si el usuario selecciona un rango de fechas (desde/hasta), se llama al servicio **`getFichajesRango(desde, hasta, IdUsuario)`** que consulta al backend **`/fichajes/rango`** con esos parámetros.

La página **`fichaje-mapa`** usa Leaflet para mostrar la ubicación de un fichaje concreto (por **`id`** en la querystring). Se carga el fichaje desde el backend con **`getFichaje(id)`**, se inicializa el mapa y se coloca un marcador.

Lógica de AppUsuarios:

Se compone principalmente de 2 páginas y el inicio.

La página **`registrar-fichaje`** toma el usuario de **`localStorage`**. Al iniciar o finalizar un fichaje, envía la petición al backend con los datos necesarios (usuario, hora, geolocalización si aplica), cuidando los tipos y validaciones. Se actualiza la imagen y cambia el estado del fichaje.

La página **`consultar-fichaje`** muestra un listado de los fichajes realizados en las últimas 24 horas indicando su estado.

Lógica del Backend:

Basado en **OpenAPI**, con archivos ``api/openapi.yaml``, ``controllers/``, ``services/`` y ``utils/openapiRouter.js`` generados. Las peticiones llegan al router dinámico según el ``openapi.yaml``; el ``DefaultController`` delega en ``DefaultService``, que implementa la lógica de acceso a DB. Los servicios usan un pool MySQL (``mysql2/promise``) y consultas parametrizadas.

Apikey:

El backend incluye un middleware ``middlewares/apikey.js`` y rutas ``routes/apikey.js`` que verifican una API key en las peticiones protegidas. Debes incluir la cabecera adecuada (por ejemplo ``x-api-key: <tu_key>``) en los endpoints que lo requieran. La integración con OpenAPI puede marcar operaciones que requieren esta key; al fallar, devuelve 401/403.

Login:

El portal en ``login/`` expone ``POST /auth/login`` que comprueba ``Usuario`` y ``Clave`` en la tabla ``Usuarios`` y retorna ``{ usuario: { IdUsuario, Nombre, Usuario, permisos } }``. El ``login.html`` guarda el usuario en ``localStorage`` y redirige según ``permisos``:

- 'D' (desarrollador) → AppAdmin.
- 'U' (usuario) → AppUsuarios.